DATA 260 Homework 10

Pragya Apurva · Rental housing listings

GitHub repository https://github.com/ishuapurva1996/data260-6102

Submission tag hw10 · The tag link resolves after GitHub publication.

Section 0 · Personal configuration

Setting	Value	Setting	Value
SID4	6102	PORT_BASE	8702
PREFIX	s6102	SEED	6102
VERIFY_SEED	266102	DOMAIN_ID	6

Hardware and models

Apple M4 MacBook Air with 24 GiB RAM, Ollama 0.33.0, and Python 3.12. Planner, Writer, Reviewer, and the DeepEval 3.9.4 G-Eval judge used local **qwen2.5:7b** (Q4_K_M). Redis semantic lookup used **nomic-embed-text:v1.5**, a 0.85 cosine cutoff and rental-data-version enforcement.

Revisions and submission evidence

Revision	Commit SHA
Verified runtime code	5ea93111e92a19452986fc1b925afad7b2aee864
HW9 base	53af679f111016ca4b6b0614274cbf11dcd88faf
Label freeze (A2)	7893184b8296862a4bfb04f9e8c6461aa89a6de8
Judge study source	6e8cceb1b6dd40ab8f19e4549aff14a54e52322f
50-miss benchmark source	d14796a39f8934965913c84fb7217f2b6dbd11c4

The runtime SHA identifies the checkout used for the live smoke. The final hw10 tag includes this PDF and its verification receipt. The local upload receipt records the exact tag target and the PDF checksum.

Setup · Run log · Metrics · Verification · AI use

What runs

A rental question first looks for a usable Redis answer. A miss passes through Planner, Writer and Reviewer over Kafka. The evaluator joins the saved messages and writes its own span under the same trace ID. Streamlit reads those saved records to show request health, answer scores and per-request timelines.

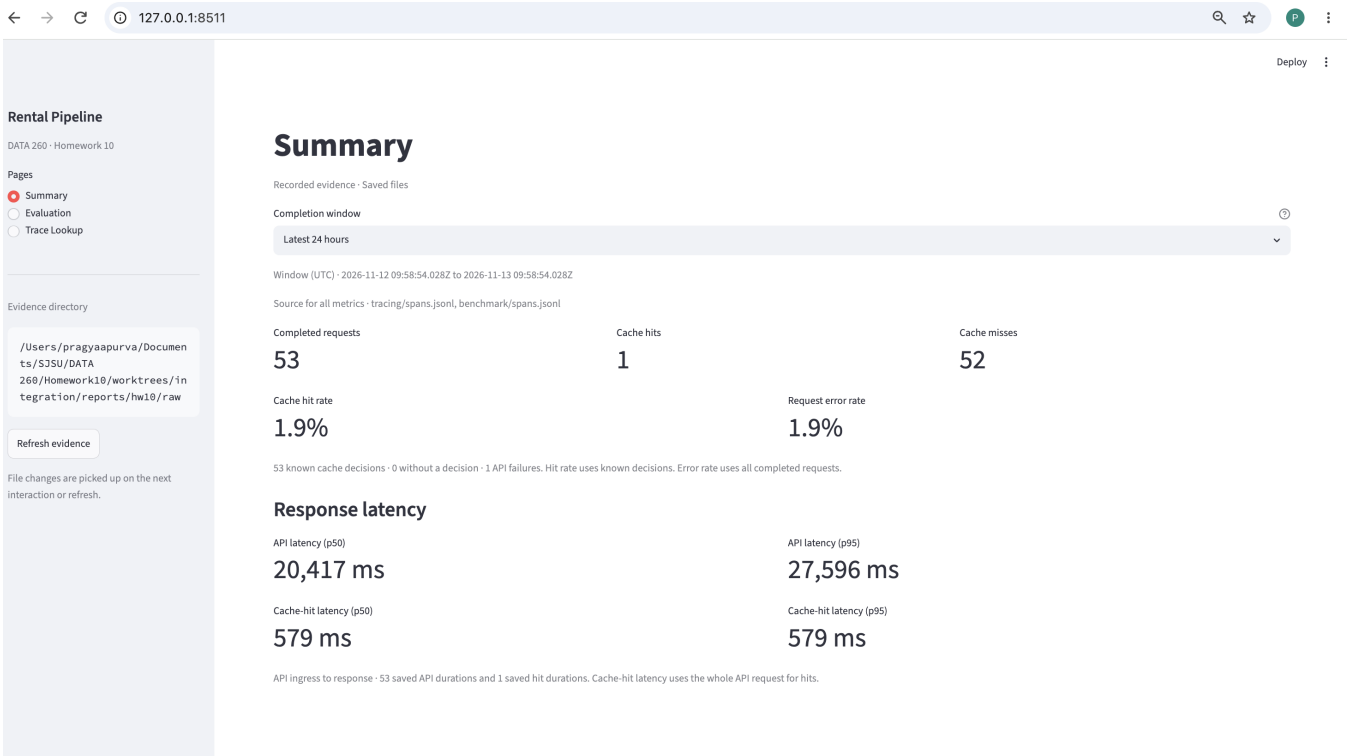
Part 1 · Streamlit Summary

Code 1. scripts/hw10/dashboard.py, lines 75–108. Count requests, classify cache outcomes and calculate both rates.

```python
def render_summary(evidence):
    spans = evidence.spans()
    requests = completed_requests(spans)
    window = window_for(requests)
    if window is None:
        return
    start, end = window
    selected = {
        trace_id: span
        for trace_id, span in requests.items()
        if start is not None and start <= span.end <= end
    }
    outcomes = cache_outcomes(spans, selected)
    hits = sum(outcomes.get(trace_id) == "hit" for trace_id in selected)
    misses = sum(outcomes.get(trace_id) == "miss" for trace_id in selected)
    errors = sum(span.api_outcome in FAILED for span in selected.values())
    sources = ", ".join(path for path in evidence.sources if path != "evaluation/spans.jsonl")
    st.caption(f"Source for all metrics · {sources or 'No saved API or cache spans'}")
    if not selected:
        st.info("No completed requests in this window.")

    metrics(
        [
            ("Completed requests", f"{len(selected):,}"),
            ("Cache hits", f"{hits:,}"),
            ("Cache misses", f"{misses:,}"),
        ]
    )
    metrics(
        [
            ("Cache hit rate", f"{hits / (hits + misses):.1%}" if hits + misses else EMPTY),
            ("Request error rate", f"{errors / len(selected):.1%}" if selected else EMPTY),
        ]
    )
```

Figure 1. Recorded 24-hour Summary and separate cache-hit latency.



Part 1 · Summary window and data source

Code excerpt. scripts/hw10/dashboard_evidence.py · lines 35–40

```
@st.cache_data(ttl=60, max_entries=32, show_spinner=False)
def parse_file(path: str, version: tuple[int, ...]) -> tuple[list[dict], int]:
    """Return usable rows and a count of invalid JSON lines or objects."""
    file = Path(path)
    text = file.read_text(encoding="utf-8")
    documents = text.splitlines() if file.suffix == ".jsonl" else [text]
```

What the window measures

The default window ends at the newest saved API completion. From November 12 09:58:54.028 UTC to November 13 09:58:54.028 UTC, 53 unique completions included one hit, 52 misses and one API error. The hit rate is hits divided by known cache decisions. The error rate uses all API completions. Both were 1.9%.

The source is the API and cache span evidence in **tracing/spans.jsonl** and **benchmark/spans.jsonl**. The all-request median was 20,417 ms and the p95 was 27,596 ms. The single hit took 578.69 ms, so both hit percentiles show that same sample. This experiment deliberately creates misses, which makes the hit rate specific to the recorded workload.

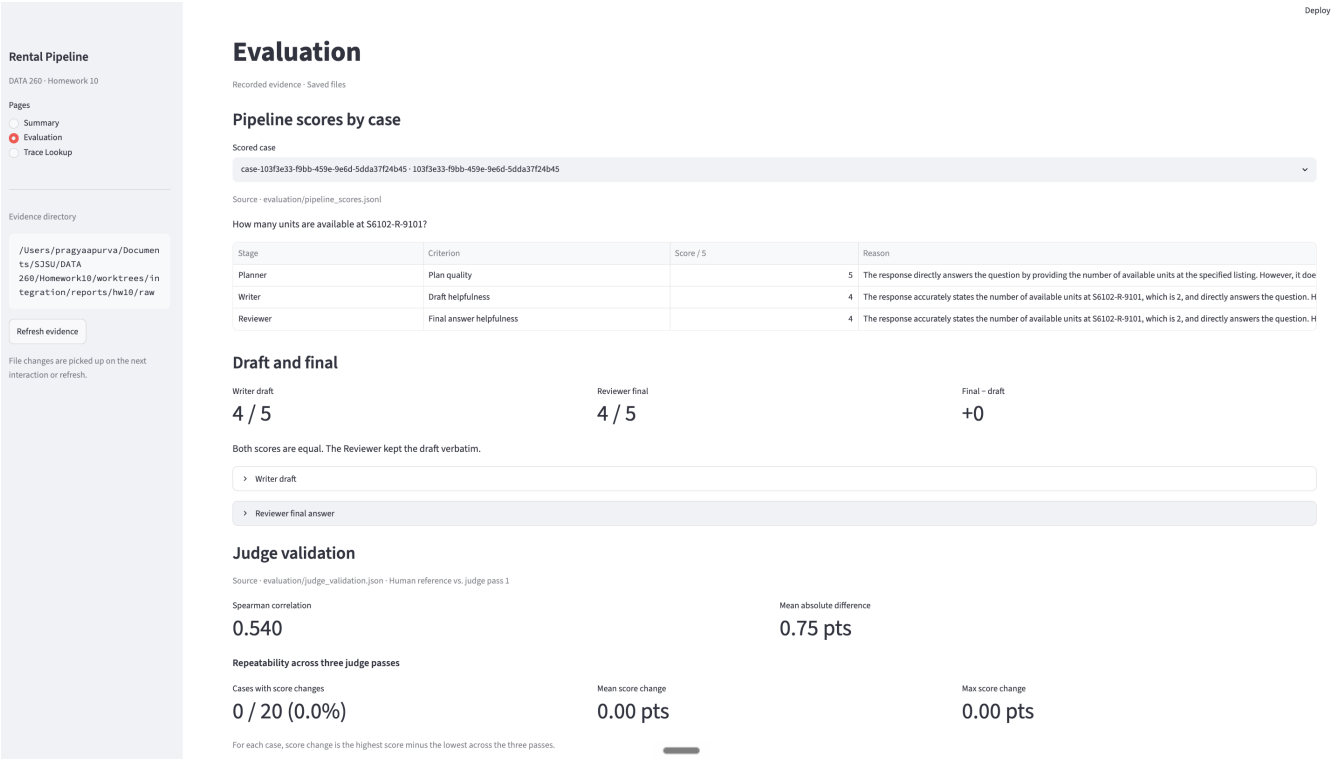
Streamlit caches file parsing for up to 60 seconds. The file version changes when the evidence changes, and Refresh clears the parsed-data cache.

Part 1 · Streamlit Evaluation

Code 2. scripts/hw10/evaluate.py, lines 80–99. Score Planner, Writer and Reviewer outputs and record the final-minus-draft score.

```python
judge = judge_factory(args)
scores = {}
for stage, criterion, field in (
    ("planner", "plan_quality", "plan"),
    ("writer", "helpfulness", "draft"),
    ("reviewer", "helpfulness", "final_answer"),
):
    scores[stage] = score_metric(
        judge, criterion, case["question"], case["reference_facts"], case[field]
    )
delta = scores["reviewer"]["score"] - scores["writer"]["score"]
result = {
    **case,
    "final": case["final_answer"],
    "evaluation_id": evaluation_id,
    "scores": scores,
    "final_minus_draft": delta,
    "interpretation": interpret(case["draft"], case["final_answer"], delta, scores),
    "judge": judge.metadata,
}
```

Figure 2. Recorded stage scores, the draft-to-final comparison, and the three-pass judge validation.



Evaluation reads the saved G-Eval output. Trace Lookup reads the combined span files and shows status and errors in start-time order (Figures 3–5).

I captured the dashboard views from the integrated app and the code from the HW10 source.

Part 2 · Evaluate the Kafka pipeline

Link the original messages

The exporter starts from the final Kafka record delivered to the API. It follows the consumed topic, partition and offset back to the inbox question, Planner plan and Writer draft, checking the shared correlation_id throughout. The scored case keeps the original payloads, rental facts and broker metadata.

Code 2 shows the stage scoring logic. Figure 2 shows the saved results.

Recorded test question

How many units are available at S6102-R-9101?

Trace ID 103f3e33-f9bb-459e-9e6d-5dda37f24b45

Stage and criterion	Normalized	Score / 5
Planner plan quality	0.90	5
Writer draft helpfulness	0.70	4
Reviewer final helpfulness	0.70	4
Final minus draft	0.00	0

Writer draft and final answer. At Cedar Court (S6102-R-9101), there are 2 units available.

The Writer carried the correct fact forward and the Reviewer kept the text verbatim, so the final had zero improvement. The Planner returned the answer directly and omitted the planning steps, despite a 5/5 plan score. The helpfulness reasons claimed the title was missing even though the text named Cedar Court. The judge credited the right unit count while missing details in both the plan and the answer it was evaluating.

Score scale and failure bounds

The evaluator rounds 1 + 4 × the normalized G-Eval score to the nearest integer with halves rounded up. A 1 is very poor or materially wrong and a 5 is accurate, relevant and complete. It gives evidence lookup 5 seconds, Ollama requests 45-second timeouts and each case 180 seconds, with up to two attempts per metric. Recorded malformed evidence and unavailable-judge checks both exited with an error span and no numeric score in under 0.1 seconds.

Part 3 · Validate the automated judge

Reference data and settings

Twenty Writer drafts came from successful cache misses. I assigned one helpfulness label to each draft before seeing judge scores. Git records the exact drafts and labels together at the freeze SHA on page 1. The validation runner verifies those bytes. The judge saw each original question, facts and draft, while the labels were used for comparison after judging.

The study ran three fresh calls per case using the same model, rubric and inputs. Qwen ran at temperature 0 and seed 6102, using JSON output, an 8,192-token context and a 256-token output limit. All 60 calls scored on the first attempt.

Code 3. scripts/hw10/evaluation_validation.py, lines 145–176. Match three judge passes with human labels by case ID and calculate the comparison metrics.

```
144
145 def compare_scores(labels, passes):
146     if (
147         not labels
148         or len(passes) != PASS_COUNT
149         or any(scores.keys() != labels.keys() for scores in passes)
150     ):
151         raise ValueError("Three complete passes must match the human labels by case_id")
152     comparisons = []
153     for identity, human in labels.items():
154         scores = [result[identity] for result in passes]
155         comparisons.append(
156             {
157                 "case_id": identity,
158                 "human_score": human,
159                 "judge_scores": scores,
160                 "pass_1_absolute_difference": abs(scores[0] - human),
161                 "score_change": max(scores) - min(scores),
162             }
163         )
164     changes = [row["score_change"] for row in comparisons]
165     changed_count = sum(change > 0 for change in changes)
166     return {
167         "spearman": spearman(list(labels.values()), [passes[0][identity] for identity in labels]),
168         "mean_absolute_difference": mean(row["pass_1_absolute_difference"] for row in comparisons),
169         "repeatability": {
170             "changed_count": changed_count,
171             "changed_rate": changed_count / len(changes),
172             "mean_score_change": mean(changes),
173             "max_score_change": max(changes),
174         },
175         "comparisons": comparisons,
176     }
177
```

Measure	Recomputed result
Spearman correlation, human vs. pass 1	0.5403
Mean absolute difference, human vs. pass 1	0.75 points
Exact matches to human ratings	7 of 20
Changed cases across three passes	0 of 20 (0%)
Mean per-case score change	0.00 points
Maximum per-case score change	0 points

Part 3 · Per-case validation results

Case ID	Human	Pass 1	Pass 2	Pass 3	|Human − P1|
hw10-01	5	4	4	4	1
hw10-02	3	4	4	4	1
hw10-03	5	5	5	5	0
hw10-04	5	5	5	5	0
hw10-05	4	2	2	2	2
hw10-06	4	4	4	4	0
hw10-07	5	4	4	4	1
hw10-08	4	4	4	4	0
hw10-09	2	2	2	2	0
hw10-10	3	4	4	4	1
hw10-11	5	4	4	4	1
hw10-12	4	4	4	4	0
hw10-13	4	4	4	4	0
hw10-14	3	4	4	4	1
hw10-15	3	4	4	4	1
hw10-16	2	4	4	4	2
hw10-17	1	2	2	2	1
hw10-18	3	4	4	4	1
hw10-19	3	4	4	4	1
hw10-20	5	4	4	4	1

All per-case changes are zero. The raw score records also preserve the trace IDs, normalized scores, run IDs, reasons and retry status. Raw judge scores.

Spearman compares the ordering of cases, using average ranks for ties. Mean absolute difference is the average point gap on the shared 1–5 scale. For repeatability, each case's change is its highest minus its lowest score across three passes.

When I would use the judge

The scores were stable under fixed settings and agreed only moderately with the human ordering. Fifteen of 20 drafts received a 4 and two ratings differed from the human label by two points. In case hw10-05, the judge repeated a 2/5 even though the answer clearly said that one available unit was fewer than two. The human score was 4/5.

I would use the judge to surface cases for review. Before using a deployment threshold, I would collect independently rated held-out cases with wrong unit counts, wrong listing IDs and invented facts. On those cases I would measure how often the threshold passes serious errors and blocks correct responses.

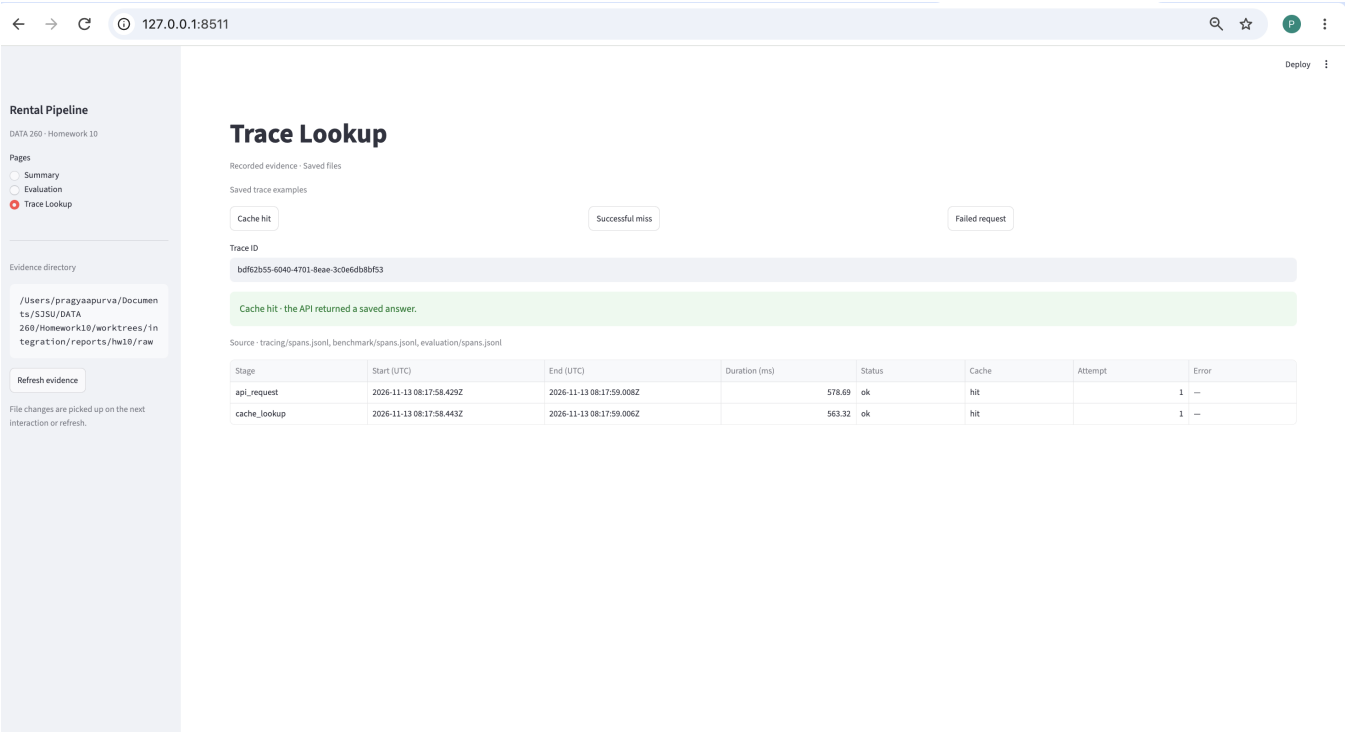
Part 4 · Trace propagation and cache hit

Every request has one ID. The API request_id appears as X-Trace-ID, Kafka correlation_id and span trace_id. The span records its stage, UTC start and end, duration, status and error when present.

Code excerpt. src/kafka_agents/transport.py · lines 18–24

```
async def send_traced(producer, topic, payload, key):
    value = json.dumps(payload).encode()
    enqueued, started = utc_now(), perf_counter()
    try:
        record = await producer.send_and_wait(
            topic, value, key=key, headers=[(ENQUEUE_HEADER, enqueued.isoformat().encode())]
        )
```

Figure 3. Cache hit. Trace bdf62b55-6040-4701-8eae-3c0e6db8bf53. The API and cache lookup are the only two spans.



The repeat used the same question and user as the successful miss. Redis returned the saved answer in 578.69 ms of API time. The cache span refers to the original miss as its source request.

Timing a Kafka handoff

The producer timestamps just before sending and places the time in a Kafka header. The consumer records receipt before decoding. The exporter links publication, receipt and acknowledgement by topic, partition and offset. The acknowledgement proves publication and can arrive after receipt.

Part 4 · Successful cache miss

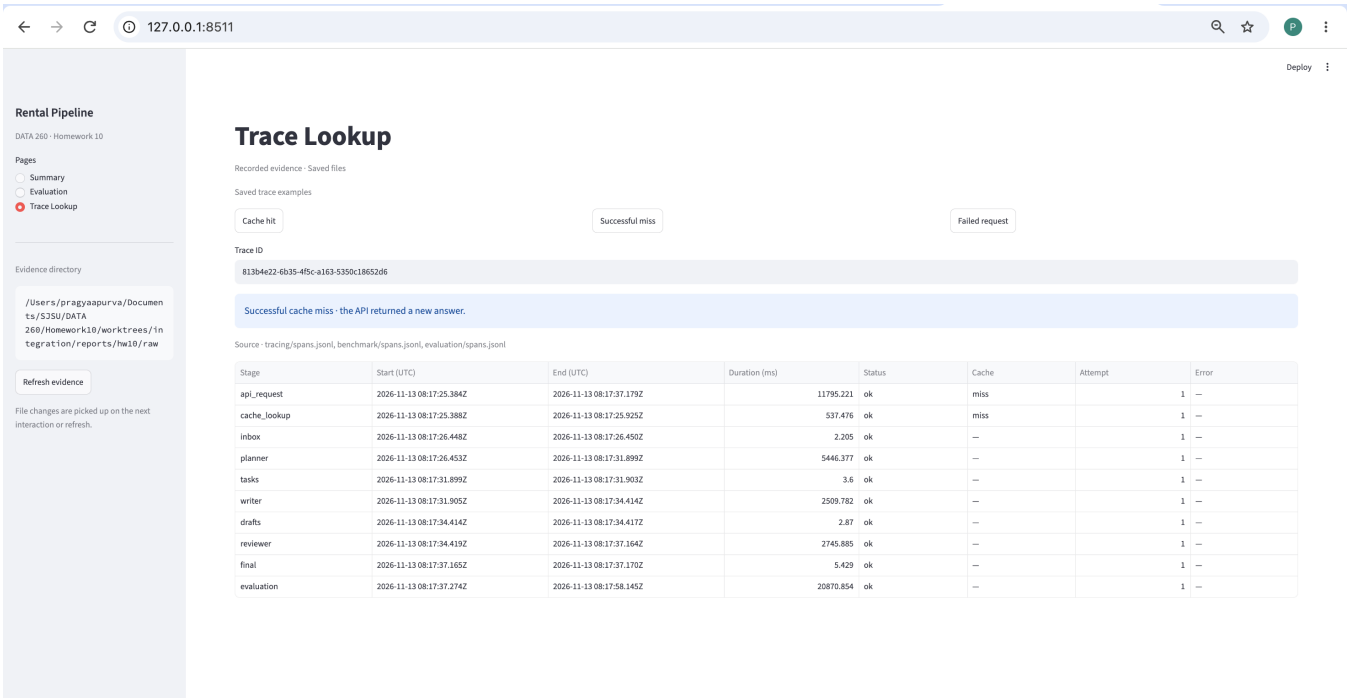
Code 4. src/kafka_agents/transport.py, lines 40–62. Record the interval from Kafka enqueue to consumer receipt under the request trace ID.

```python
    return record, {"enqueue_time": enqueued.isoformat(), "ack_time": utc_now().isoformat()}


def trace_receipt(record, trace_id, received, *, error=None):
    value = dict(getattr(record, "headers", None) or ()).get(ENQUEUE_HEADER)
    if not value:
        return  # Older messages have no measured enqueue time.
    try:
        enqueued = datetime.fromisoformat(value.decode("ascii"))
    except (ValueError, UnicodeError):
        return
    if enqueued.utcoffset() != timedelta(0):
        return
    duration_ms = (received - enqueued).total_seconds() * 1000
    trace_span(
        trace_id,
        HANDOFFS[record.topic],
        enqueued,
        received,
        duration_ms if duration_ms >= 0 else None,
        operation="queue_wait",
        enqueue_time=enqueued.isoformat(),
        receive_time=received.isoformat(),
        error=error or ("Kafka receipt clock precedes enqueue" if duration_ms < 0 else None),
        **coordinates(record),
    )

```

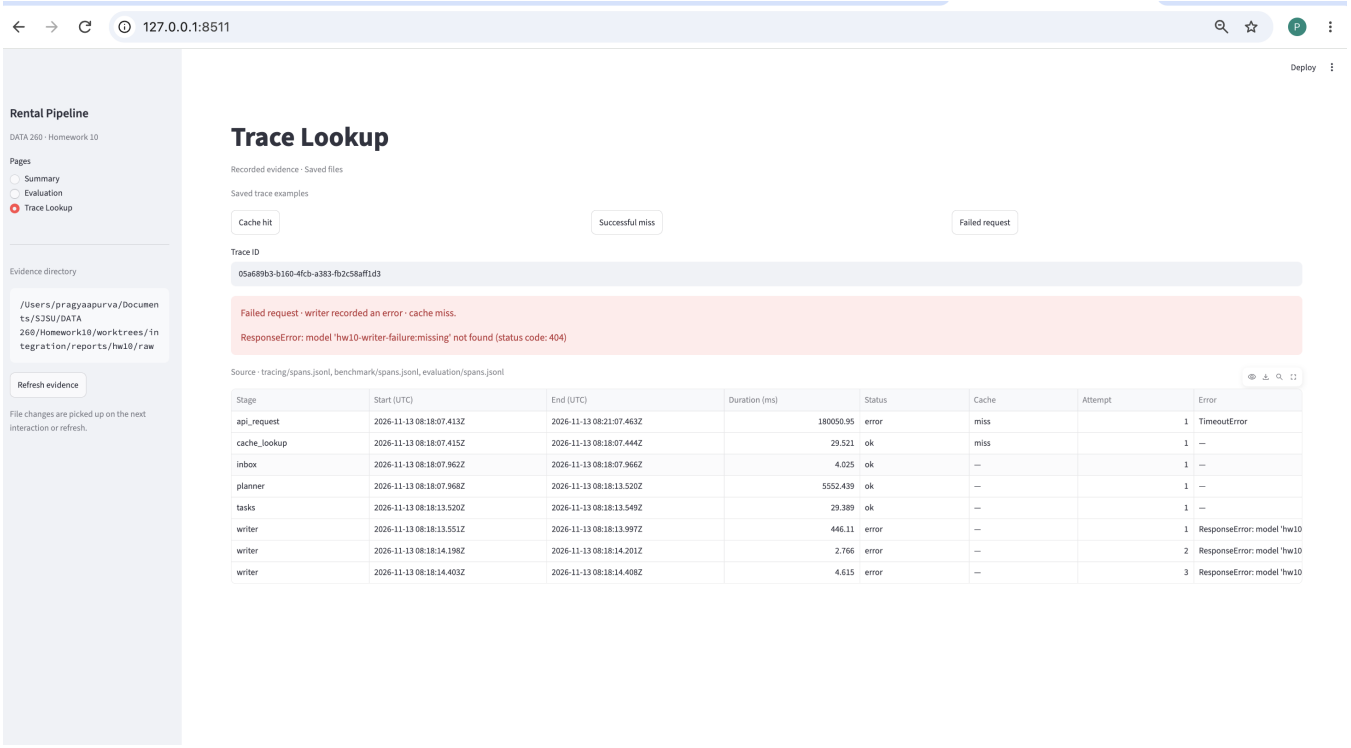
Figure 4. Successful miss. Trace 813b4e22-6b35-4f5c-a163-5350c18652d6. Ten spans cover API ingress, cache lookup, Kafka delivery, all three agents and evaluation.



The example returned HTTP 200 in 11,795.22 ms. Evaluation began once the linked messages were exported. API latency ends at the response.

Part 4 · Controlled Writer failure

Figure 5. Failed request. Trace 05a689b3-b160-4fcb-a383-fb2c58aff1d3. The Writer error appears in the banner and the span table.



Error and recovery

The cache lookup completed as a miss. Ollama returned HTTP 404 because the deliberately configured Writer model **hw10-writer-failure:missing** was absent. The Writer made three attempts and published the failure to **s6102_dead_letter**, partition 0, offset 0. The API returned HTTP 504 after waiting about 180 seconds for a final answer.

The healthy Writer model and original consumer group were restored. The same question then returned HTTP 200 as a miss under trace **ce24f3d8-608d-4e7f-a834-7953c5acd6d0**. Example receipts.

Part 4 · Fifty confirmed cache misses

Code 5. scripts/hw10/benchmark_timing.py, lines 206–237. Calculate p50, p95 and each stage's share of full-trace latency.

```python

def distribution(rows, key):
    values = [row[key] for row in rows]
    return {"p50_ms": percentile(values, 0.5), "p95_ms": percentile(values, 0.95)}


def timing_summary(rows):
    if len({row["trace_id"] for row in rows}) != len(rows):
        raise EvidenceError("duplicate trace IDs in timing rows")
    if not rows:
        raise EvidenceError("no admitted traces to summarize")

    total_ms = math.fsum(row["full_trace_ms"] for row in rows)
    stages = [
        {
            "stage": stage,
            **distribution(rows, f"{stage}_ms"),
            "percent_of_full_trace": (
                100 * math.fsum(row[f"{stage}_ms"] for row in rows) / total_ms
            ),
        }
        for stage in STAGES
    ]
    shares = math.fsum(row["percent_of_full_trace"] for row in stages)
    return {
        "stages": stages,
        "api_latency": distribution(rows, "api_latency_ms"),
        "http_latency": distribution(rows, "http_latency_ms"),
        "full_trace": distribution(rows, "full_trace_ms"),
        "full_trace_total_ms": total_ms,
        "stage_share_total_percent": shares,
        "unallocated_or_overlapping_percent": 100 - shares,
        "dominant_stage": max(stages, key=lambda row: row["percent_of_full_trace"])["stage"],
    }
```

Stage	p50 (ms)	p95 (ms)	% full trace
Cache lookup	547.11	824.75	1.03%
Planner	7,192.90	10,559.49	13.13%
Writer	5,640.52	7,880.57	10.26%
Reviewer	6,198.59	8,509.46	11.08%
Queue wait (total)	16.55	63.25	0.05%
Evaluation	34,246.01	43,386.94	62.98%

API ingress to response was **20,489.16 ms p50** and **27,301.06 ms p95**. Ingress to evaluation completion was **54,400.97 ms p50** and **71,289.31 ms p95**. Each request's queue time sums its four enqueue-to-receipt intervals before percentiles. Shares divide the sum of a stage's time by the sum of full-trace time across the same 50 requests. Fact retrieval, cache writes and other untimed work account for the remaining 1.47%.

Part 4 · Benchmark method and interpretation

Code excerpt. scripts/hw10/benchmark_timing.py · lines 96–104

```
def timing_row(attempt, spans):
    if attempt.status_code != 200 or attempt.error or not attempt.trace_id:
        raise EvidenceError("missing successful HTTP response with request_id")
    if attempt.is_cached is not False:
        raise EvidenceError("API response is_cached must be false")
    spans = unique_spans(span for span in spans if span.get("trace_id") == attempt.trace_id)
    api, ingress, response_end, api_latency_ms = completed_span(spans, "api_request")
    if api.get("cache_outcome") != "miss":
        raise EvidenceError("api_request did not record a miss")
```

The warmed stack ran one request at a time across 50 distinct questions on ten synthetic listings. An empty owned cache namespace forced each question through normal lookup. Every admitted row matched HTTP 200 with is_cached=false, two completed miss records, four Kafka handoffs and successful evaluation. Seven of 57 attempts were excluded, including the warm-up, clock interruptions, an overlapped request and a surplus repeat.

What I would optimize first

Evaluation accounted for 62.98% of full-trace time. Ollama logged reloads when switching between 4,096-token agent context and 8,192-token judge context. I would test a shared context and compare time and answer quality on the same cases. Planner time was the largest agent component. Queue wait was 0.05% of the full trace, so it offers little room for improvement on this workload.

AI use and final verification

1. AI assistance and my work

I gave Codex the assignment and integration scope, supplied the exact milestone commits, and personally assigned the helpfulness labels for all 20 Writer drafts before seeing any judge scores. I also took the dashboard and code screenshots. Codex assisted with the tracing, evaluator, dashboard, benchmark, evidence checks, integration and report preparation. The local Qwen model served as both agent generator and G-Eval judge. The human labels were my own work.

2. One unsuitable AI-produced output

For case hw10-05, the judge gave the draft 2/5 and said it failed to answer whether the listing had at least two units. The draft explicitly answered no and stated that one unit was available. The frozen human label was 4/5.

3. How the issue was detected

The evaluation audit joined the raw judge scores and frozen human labels by case ID. It compared the reason with the unchanged draft and listing facts. The same 2/5 verdict appeared in all three passes.

4. What changed and why

The report uses the judge as a diagnostic alongside human review and keeps both scores in the evidence so the disagreement can be audited. A deployment gate would require independent ratings and tests for serious factual errors.

Integrated check

On the joined code, an authenticated question produced ten spans through G-Eval. The exact repeat produced two spans and the same answer. Both returned HTTP 200. The integrated verifier reopened the three recorded example traces and recomputed both measured tables. The 109 application, evaluation and benchmark tests and eight Streamlit tests passed.

Live smoke	Trace ID
Miss + evaluation	442e41c3-20fc-477d-953a-0740c591fe39
Exact repeat hit	141f70f9-d40f-4e56-987b-f60922c502d9

The study and benchmark retain their measured source revisions. The integrated runtime matches the measured execution files and admits the same 50 measured traces with the recorded clock consistency check.